Nested Varying Effects

General Principles

To model hierarchies where groups are themselves nested within larger units (e.g., students within classes, which are within schools), we use *Nested Varying Effects*. This allows the model to share information across different levels of the hierarchy, improving estimates through multilevel pooling.

Note

> **Note**
>
> - We have the same considerations as for Varying Intercepts and Varying Slopes.
> - In a nested model, a parameter at one level (e.g., a group-level intercept) becomes the mean for the distribution of parameters at a lower level (e.g., individual-level intercepts).
> - This structure is often represented using multiple indices: j for the group and $k(j)$ for the super-group (e.g., region) that group j belongs to.
> - To capture the correlation between multiple varying effects (e.g., intercept and slope) at any level of the hierarchy, we use a **Multivariate Normal (MVN)** distribution.
> - Non-centered parameterization is highly recommended for nested models to avoid "funnel" geometries that can hinder MCMC sampling.

Example

Below is an example of a nested model with both **varying intercepts** and **varying slopes**. We model the outcome y for individuals in `groups` nested within `regions`. The relationship between x and y varies at both levels.

Python (Centered)

```python
from BayesForge import bf
import jax.numpy as jnp
import numpy as np

# Setup device ------------------------------------------------
m = bf(platform='cpu')

# Load data - column args are mapped automatically from the DataFrame.
# N_groups and N_regions are dataset-specific; provide them as defaults.
data_path = m.load.sim_nested_effects(only_path=True)
m.data(data_path)

# Define model ------------------------------------------------
def model_nested(y, x, group_id, region_id, N_groups=20, N_regions=5):
    sigma = m.dist.exponential(1, name='sigma')

    # 1. Region level
    mu_global = jnp.stack([m.dist.normal(5, 2,  name='global_intercept'),
                           m.dist.normal(-1, 1, name='global_beta')])
    sigma_reg  = m.dist.exponential(1, shape=(2,), name='sigma_region')
    corr_reg   = m.dist.lkj(2, 2, name='corr_region')
    cov_reg    = jnp.diag(sigma_reg) @ corr_reg @ jnp.diag(sigma_reg)
    region_effects = m.dist.multivariate_normal(
        mu_global, cov_reg, shape=(N_regions,), name='region_effects'
    )

    # 2. Group level - parent mapping via JAX scatter (traceable under pmap)
    group_to_region = jnp.zeros(N_groups, dtype=region_id.dtype).at[group_id].set(region_id)
    sigma_grp  = m.dist.exponential(1, shape=(2,), name='sigma_group')
    corr_grp   = m.dist.lkj(2, 2, name='corr_group')
    cov_grp    = jnp.diag(sigma_grp) @ corr_grp @ jnp.diag(sigma_grp)
    group_effects = m.dist.multivariate_normal(
        region_effects[group_to_region], cov_grp, name='group_effects'
    )

    mu_est = group_effects[group_id, 0] + group_effects[group_id, 1] * x
    m.dist.normal(mu_est, sigma, obs=y)

# Run sampler ------------------------------------------------
m.fit(model_nested, num_samples=1000, num_warmup=500, num_chains=1)
```

```
m.summary()
```

```
An NVIDIA GPU may be present on this machine, but a CUDA-enabled jaxlib is not installed. Fal

bf v 0.0.60 package loaded
jax.local_device_count 16

  0%|          | 0/1500 [00:00<?, ?it/s]warmup:   5%|          | 75/1500 [00:06<02:04, 11.42i
```

	mean	sd	eti89_lb	eti89_ub	ess_bulk	ess_tail	r_hat	mcse_mean	mcse_sd
corr_group[0, 0]	1.00	0.00	1.00	1.00	1000.00	1000.00	NaN	0.00	NaN
corr_group[0, 1]	0.42	0.22	0.03	0.73	676.41	620.35	NaN	0.01	0.01
corr_group[1, 0]	0.42	0.22	0.03	0.73	676.41	620.35	NaN	0.01	0.01
corr_group[1, 1]	1.00	0.00	1.00	1.00	925.67	935.45	NaN	0.00	0.00
corr_region[0, 0]	1.00	0.00	1.00	1.00	1000.00	1000.00	NaN	0.00	NaN
...	...	...	...	...	...	...	...	...	...
sigma	0.49	0.02	0.46	0.52	1935.65	722.28	NaN	0.00	0.00
sigma_group[0]	0.42	0.08	0.31	0.56	699.92	626.27	NaN	0.00	0.00
sigma_group[1]	0.25	0.06	0.16	0.36	542.78	794.35	NaN	0.00	0.00
sigma_region[0]	1.02	0.39	0.57	1.77	792.90	812.67	NaN	0.01	0.02
sigma_region[1]	0.47	0.26	0.21	0.88	634.42	704.07	NaN	0.01	0.03

Python (Non-Centered)

```python
# Data and m object carry over from the Centered tab above.

def model_nested_noncentered(y, x, group_id, region_id, N_groups=20, N_regions=5):
    sigma = m.dist.exponential(1, name='sigma')

    # 1. Region level
    mu_global    = jnp.stack([m.dist.normal(5, 2,  name='global_intercept'),
                              m.dist.normal(-1, 1, name='global_beta')])
    sigma_region = m.dist.exponential(1, shape=(2,), name='sigma_region')
    L_region     = m.dist.lkj_cholesky(2, 2, name='L_region')
    z_region     = m.dist.normal(0, 1, name='z_region', shape=(2, N_regions))
    region_effects = mu_global + ((sigma_region[..., None] * L_region) @ z_region).T

    # 2. Group level - parent mapping via JAX scatter (traceable under pmap)
```

```
    pid           = jnp.zeros(N_groups, dtype=region_id.dtype).at[group_id].set(region_id)
    sigma_group   = m.dist.exponential(1, shape=(2,), name='sigma_group')
    L_group       = m.dist.lkj_cholesky(2, 2, name='L_group')
    z_group       = m.dist.normal(0, 1, name='z_group', shape=(2, N_groups))
    group_effects = region_effects[pid] + ((sigma_group[..., None] * L_group) @ z_group).T

    mu_est = group_effects[group_id, 0] + group_effects[group_id, 1] * x
    m.dist.normal(mu_est, sigma, obs=y)

m.fit(model_nested_noncentered, num_samples=1000, num_warmup=500, num_chains=1)
m.summary()
```

```
  0%|          | 0/1500 [00:00<?, ?it/s]warmup:   5%|          | 75/1500 [00:10<03:14,  7.33i
```

	mean	sd	eti89_lb	eti89_ub	ess_bulk	ess_tail	r_hat	mcse_mean	mcse_sd
L_group[0, 0]	1.00	0.00	1.00	1.00	1000.00	1000.00	NaN	0.00	NaN
L_group[0, 1]	0.00	0.00	0.00	0.00	1000.00	1000.00	NaN	0.00	NaN
L_group[1, 0]	0.42	0.24	0.00	0.76	702.65	673.83	NaN	0.01	0.01
L_group[1, 1]	0.87	0.11	0.65	1.00	722.56	673.83	NaN	0.00	0.00
L_region[0, 0]	1.00	0.00	1.00	1.00	1000.00	1000.00	NaN	0.00	NaN
...	...	...	...	...	...	...	...	...	...
z_region[1, 0]	-0.70	0.65	-1.70	0.35	768.24	636.97	NaN	0.02	0.02
z_region[1, 1]	1.03	0.67	0.02	2.04	920.69	648.91	NaN	0.02	0.02
z_region[1, 2]	-0.60	0.67	-1.64	0.44	732.62	571.41	NaN	0.03	0.02
z_region[1, 3]	0.36	0.75	-0.83	1.57	1087.86	762.23	NaN	0.02	0.02
z_region[1, 4]	0.02	0.68	-1.07	1.15	904.28	693.38	NaN	0.02	0.02

Python (Built-in)

```
from BayesForge import bf
import jax.numpy as jnp

# Setup device -------------------------------------------------
m = bf(platform='cpu')

# Load data - column args are mapped automatically from the DataFrame.
# N_groups and N_regions are dataset-specific; provide them as defaults.
data_path = m.load.sim_nested_effects(only_path=True)
m.data(data_path)
```

```python
# group_ids: one obs-level index array per level, top to bottom.
# Parent structure is derived automatically from these arrays.
def model_nested_builtin(y, x, group_id, region_id, N_groups=20, N_regions=5):
    sigma = m.dist.exponential(1, name='sigma')

    a_g_est, b_g_est = m.effects.nested_varying_effects(
        N_vars=2,
        names=["region", "group"],
        N_groups=[N_regions, N_groups],
        group_ids=[region_id, group_id],
        centered=False,
    )

    mu_est = a_g_est + b_g_est * x
    m.dist.normal(mu_est, sigma, obs=y)

m.fit(model_nested_builtin, num_samples=1000, num_warmup=500, num_chains=1)
m.summary()
```

```
jax.local_device_count 16

  0%|          | 0/1500 [00:00<?, ?it/s]warmup:   0%|          | 0/1500 [00:13<?, ?it/s, 127
```

	mean	sd	eti89_lb	eti89_ub	ess_bulk	ess_tail	r_hat	mcse_mean	mcse_s
L_corr_group[0, 0]	1.00	0.00	1.00	1.00	1000.00	1000.00	NaN	0.00	NaN
L_corr_group[0, 1]	0.00	0.00	0.00	0.00	1000.00	1000.00	NaN	0.00	NaN
L_corr_group[1, 0]	0.43	0.22	0.04	0.75	655.32	625.13	NaN	0.01	0.01
L_corr_group[1, 1]	0.87	0.11	0.67	1.00	626.37	630.74	NaN	0.00	0.00
L_corr_region[0, 0]	1.00	0.00	1.00	1.00	1000.00	1000.00	NaN	0.00	NaN
...	...	...	...	...	...	...	...	...	...
z_region[1, 0]	-0.71	0.60	-1.69	0.21	791.68	687.75	NaN	0.02	0.02
z_region[1, 1]	0.94	0.70	-0.13	2.08	652.61	715.61	NaN	0.03	0.02
z_region[1, 2]	-0.65	0.65	-1.65	0.42	841.23	709.81	NaN	0.02	0.02
z_region[1, 3]	0.31	0.74	-0.91	1.46	930.47	789.45	NaN	0.02	0.02
z_region[1, 4]	-0.08	0.72	-1.20	1.10	752.63	585.32	NaN	0.03	0.02

Python (Built-in Centered)

```python
from BayesForge import bf
import jax.numpy as jnp

# Setup device -------------------------------------------------
m = bf(platform='cpu')

# Load data - column args are mapped automatically from the DataFrame.
# N_groups and N_regions are dataset-specific; provide them as defaults.
data_path = m.load.sim_nested_effects(only_path=True)
m.data(data_path)

def model_nested_builtin_centered(y, x, group_id, region_id, N_groups=20, N_regions=5):
    sigma = m.dist.exponential(1, name='sigma')

    a_g_est, b_g_est = m.effects.nested_varying_effects(
        N_vars=2,
        names=["region", "group"],
        N_groups=[N_regions, N_groups],
        group_ids=[region_id, group_id],
        centered=True,
    )

    mu_est = a_g_est + b_g_est * x
    m.dist.normal(mu_est, sigma, obs=y)

m.fit(model_nested_builtin_centered, num_samples=1000, num_warmup=500, num_chains=1)
m.summary()
```

```
jax.local_device_count 16

  0%|          | 0/1500 [00:00<?, ?it/s]warmup:   5%|          | 75/1500 [00:09<03:08,  7.55i
```

	mean	sd	eti89_lb	eti89_ub	ess_bulk	ess_tail	r_hat	mcse_mean	mcse_sd
corr_group[0, 0]	1.00	0.00	1.00	1.00	1000.00	1000.00	NaN	0.00	NaN
corr_group[0, 1]	0.41	0.23	0.01	0.75	683.54	700.64	NaN	0.01	0.01
corr_group[1, 0]	0.41	0.23	0.01	0.75	683.54	700.64	NaN	0.01	0.01
corr_group[1, 1]	1.00	0.00	1.00	1.00	958.78	829.05	NaN	0.00	0.00
corr_region[0, 0]	1.00	0.00	1.00	1.00	1000.00	1000.00	NaN	0.00	NaN
...	...	...	...	...	...	...	...	...	...

	mean	sd	eti89_lb	eti89_ub	ess_bulk	ess_tail	r_hat	mcse_mean	mcse_sd
sigma	0.49	0.02	0.46	0.53	1417.57	694.97	NaN	0.00	0.00
sigma_group[0]	0.42	0.09	0.30	0.57	656.41	607.51	NaN	0.00	0.00
sigma_group[1]	0.25	0.07	0.17	0.37	600.14	618.51	NaN	0.00	0.00
sigma_region[0]	1.10	0.47	0.58	1.96	980.90	896.03	NaN	0.02	0.02
sigma_region[1]	0.47	0.23	0.21	0.90	685.27	641.27	NaN	0.01	0.01

Julia

```
using PyBayesForge

# Setup device ------------------------------------------------
m = importBF(platform="cpu")

# Load data - column args mapped automatically from the DataFrame.
# N_groups and N_regions are dataset-specific; provide them as defaults.
data_path = m.load.sim_nested_effects(only_path=true)
m.data(data_path)

# The Julia wrapper exposes no Python-visible defaults, so pass the model
# data (DataFrame columns + scalar sizes) explicitly.
m.data_on_model = pydict(
    y         = jnp.array(m.df["y"]),
    x         = jnp.array(m.df["x"]),
    group_id  = jnp.array(m.df["group_id"]),
    region_id = jnp.array(m.df["region_id"]),
    N_groups  = 20,
    N_regions = 5,
)

# Define model ------------------------------------------------
@BF function model(y, x, group_id, region_id, N_groups=20, N_regions=5)
    sigma = m.dist.exponential(1, name="sigma")

    veff = m.effects.nested_varying_effects(
        N_vars    = 2,
        names     = ["region", "group"],
        N_groups  = [N_regions, N_groups],
        group_ids = [region_id, group_id],
        centered  = false,
```

```
    )
    a_g_est, b_g_est = veff

    mu = a_g_est + b_g_est * x
    m.dist.normal(mu, sigma, obs=y, name="y")
end

# Run mcmc ------------------------------------------------
m.fit(model, num_samples=1000, num_warmup=500, num_chains=1)
m.summary()
```

R

```
library(BayesForge)

# Setup platform ------------------------------------------------
m = importBF(platform='cpu')
jnp = reticulate::import("jax.numpy")

# Load data - column args mapped automatically from the DataFrame.
# N_groups and N_regions are dataset-specific; provide them as defaults.
data_path = m$load$sim_nested_effects(only_path=TRUE)
m$data(data_path)

# The R wrapper exposes no Python-visible defaults, so pass the model
# data (DataFrame columns + scalar sizes) explicitly.
m$data_on_model = list(
  y         = jnp$array(m$df$y),
  x         = jnp$array(m$df$x),
  group_id  = jnp$array(as.integer(m$df$group_id)),
  region_id = jnp$array(as.integer(m$df$region_id)),
  N_groups  = 20L,
  N_regions = 5L
)

# Define model ------------------------------------------------
model <- function(y, x, group_id, region_id, N_groups=20L, N_regions=5L) {
  sigma = m$dist$exponential(1, name='sigma')

  veff = m$effects$nested_varying_effects(
    N_vars    = 2L,
```

```
    names     = list('region', 'group'),
    N_groups  = list(as.integer(N_regions), as.integer(N_groups)),
    group_ids = list(region_id, group_id),
    centered  = FALSE,
  )
  a_g_est = veff[[1]]
  b_g_est = veff[[2]]

  mu = a_g_est + b_g_est * x
  m$dist$normal(mu, sigma, obs=y, name='y')
}

# Run MCMC ------------------------------------------------
m$fit(model, num_samples=1000L, num_warmup=500L, num_chains=1L)
m$summary()
```

Mathematical Details

We model the outcome Y_i for observation i. The intercept α_j and slope β_j for group $j(i)$ are nested within region $k(j)$.

$$Y_i \sim \text{Normal}(\mu_i, \sigma)$$

$$\mu_i = \alpha_{[j(i)]} + \beta_{[j(i)]} X_i$$

Centered Parameterization

The vector of group-level effects follows a Multivariate Normal distribution centered on the region-level effects:

$$\begin{bmatrix} \alpha_j \\ \beta_j \end{bmatrix} \sim \text{MVNormal}\left(\begin{bmatrix} \gamma_{\alpha,[k(j)]} \\ \gamma_{\beta,[k(j)]} \end{bmatrix}, \Sigma_{group} \right)$$

The region-level effects themselves follow their own Multivariate Normal distribution:

$$\begin{bmatrix} \gamma_{\alpha,k} \\ \gamma_{\beta,k} \end{bmatrix} \sim \text{MVNormal}\left(\begin{bmatrix} \bar{\gamma}_{\alpha} \\ \bar{\gamma}_{\beta} \end{bmatrix}, \Sigma_{region} \right)$$

Where: - $\Sigma = \mathbf{SRS}$ (diagonal matrix of standard deviations $\mathbf{S}$ and correlation matrix $\mathbf{R}$). - $\bar{\gamma}_\alpha, \bar{\gamma}_\beta$ are the global average intercept and slope. - $\Sigma_{region}, \Sigma_{group}$ are the covariance matrices for each hierarchical level.

Non-Centered Parameterization

To avoid funnel geometries, the non-centered formulation instead uses Cholesky factors L_{region} and L_{group} alongside standardized, non-correlated normal variables:

$$\begin{bmatrix} z_{\alpha,k} \\ z_{\beta,k} \end{bmatrix} \sim \text{Normal}(0, 1), \quad \begin{bmatrix} z_{\alpha,j} \\ z_{\beta,j} \end{bmatrix} \sim \text{Normal}(0, 1)$$

The region-level and group-level distributions are linearly transformed:

$$\begin{bmatrix} \gamma_{\alpha,k} \\ \gamma_{\beta,k} \end{bmatrix} = \begin{bmatrix} \bar{\gamma}_\alpha \\ \bar{\gamma}_\beta \end{bmatrix} + \text{diag}(\mathbf{S}_{region}) L_{region} \begin{bmatrix} z_{\alpha,k} \\ z_{\beta,k} \end{bmatrix}$$

$$\begin{bmatrix} \alpha_j \\ \beta_j \end{bmatrix} = \begin{bmatrix} \gamma_{\alpha,[k(j)]} \\ \gamma_{\beta,[k(j)]} \end{bmatrix} + \text{diag}(\mathbf{S}_{group}) L_{group} \begin{bmatrix} z_{\alpha,j} \\ z_{\beta,j} \end{bmatrix}$$

Model validation tests

- Parameter recovery: `builtin_function.py` simulates nested data with known parameters, fits the built-in nested model and checks that the parameters are recovered.
- Documentation examples: the Python, R and Julia examples of this page are run by the documentation tests (`Python/test_model_pages.py`, `R/test_r_model_pages.py`, `Julia/test_julia_model_pages.py`).

Notes

> **Note**
>
> - By nesting the **MVN** prior, we allow the model to learn how the correlation between intercepts and slopes persists across different levels of the hierarchy.
> - The non-centered parameterization (used by default in the `m.effects.nested_varying_effects` built-in function) is critical for convergence in these complex models.
> - This approach can be generalized to N levels and M variables by increasing the dimensionality of the vectors and matrices.

Reference(s)